

ASCII code BIN 출력하기 과제

source code

...

```
#include <stdio.h>

void print_bin(int n) {
    char bin[8];
    int i;
    for (i = 7; i >= 0; i--) {
        bin[i] = '0' + (n % 2);
        n /= 2;
    }
    for (i = 0; i < 8; i++)
        printf("%c", bin[i]);
}

int main(void) {
    char* name[] = {
        "NUL", "SOH", "STX", "ETX", "EOT", "ENQ", "ACK", "BEL",
        "BS", "TAB", "LF", "VT", "FF", "CR", "SO", "SI",
        "DLE", "DC1", "DC2", "DC3", "DC4", "NAK", "SYN", "ETB",
        "CAN", "EM", "SUB", "ESC", "FS", "GS", "RS", "US",
        "SP"
    };

    printf("          ASCII CODE TABLE\n");
    printf("          =====\n");
    printf("DEC  HEX  BIN      CHAR DEC  HEX  BIN      CHAR DEC  HEX  BIN\n");
    printf("CHAR DEC  HEX  BIN      CHAR\n");
    printf("====  ==  =====  ==  ==  =====  ==  ==  =====\n");
    printf("=====  ==  == =====  =====\n");

    int i;
    for (i = 0; i < 32; i++) {
        // 1열: 0~31
        printf("%3d %02X ", i, i);
        print_bin(i);
        printf(" %-4s ", name[i]);

        // 2열: 32~63
        printf("%3d %02X ", i + 32, i + 32);
        print_bin(i + 32);
        printf(" %-4s ", i + 32 == 32 ? name[32] : (char[]) { i + 32, 0 });

        // 3열: 64~95
        printf("%3d %02X ", i + 64, i + 64);
        print_bin(i + 64);
        printf(" %-4s ", (char[]) { i + 64, 0 });

        // 4열: 96~127
        printf("%3d %02X ", i + 96, i + 96);
        print_bin(i + 96);
    }
}
```

```

        if (i + 96 == 127)
            printf("  DELWn");
        else
            printf("  %-4sWn", (char[]) { i + 96, 0 });
    }

    return 0;
}
...

```

실행결과

ASCII CODE TABLE															
=====															
DEC	HEX	BIN	CHAR	DEC	HEX	BIN	CHAR	DEC	HEX	BIN	CHAR	DEC	HEX	BIN	CHAR
----	----	-----	----	----	----	-----	----	----	----	-----	----	----	----	-----	----
0	00	00000000	NUL	32	20	00100000	SP	64	40	01000000	@	96	60	01100000	`
1	01	00000001	SOH	33	21	00100001	!	65	41	01000001	A	97	61	01100001	a
2	02	00000010	STX	34	22	00100010	"	66	42	01000010	B	98	62	01100010	b
3	03	00000011	ETX	35	23	00100011	#	67	43	01000011	C	99	63	01100011	c
4	04	00000100	EOT	36	24	00100100	\$	68	44	01000100	D	100	64	01100100	d
5	05	00000101	ENQ	37	25	00100101	%	69	45	01000101	E	101	65	01100101	e
6	06	00000110	ACK	38	26	00100110	&	70	46	01000110	F	102	66	01100110	f
7	07	00000111	BEL	39	27	00100111	'	71	47	01000111	G	103	67	01100111	g
8	08	00001000	BS	40	28	00101000	(	72	48	01001000	H	104	68	01101000	h
9	09	00001001	TAB	41	29	00101001	)	73	49	01001001	I	105	69	01101001	i
10	0A	00001010	LF	42	2A	00101010	*	74	4A	01001010	J	106	6A	01101010	j
11	0B	00001011	VT	43	2B	00101011	+	75	4B	01001011	K	107	6B	01101011	k
12	0C	00001100	FF	44	2C	00101100	,	76	4C	01001100	L	108	6C	01101100	l
13	0D	00001101	CR	45	2D	00101101	-	77	4D	01001101	M	109	6D	01101101	m
14	0E	00001110	SO	46	2E	00101110	.	78	4E	01001110	N	110	6E	01101110	n
15	0F	00001111	SI	47	2F	00101111	/	79	4F	01001111	O	111	6F	01101111	o
16	10	00010000	DLE	48	30	00110000	0	80	50	01010000	P	112	70	01110000	p
17	11	00010001	DC1	49	31	00110001	1	81	51	01010001	Q	113	71	01110001	q
18	12	00010010	DC2	50	32	00110010	2	82	52	01010010	R	114	72	01110010	r
19	13	00010011	DC3	51	33	00110011	3	83	53	01010011	S	115	73	01110011	s
20	14	00010100	DC4	52	34	00110100	4	84	54	01010100	T	116	74	01110100	t
21	15	00010101	NAK	53	35	00110101	5	85	55	01010101	U	117	75	01110101	u
22	16	00010110	SYN	54	36	00110110	6	86	56	01010110	V	118	76	01110110	v
23	17	00010111	ETB	55	37	00110111	7	87	57	01010111	W	119	77	01110111	w
24	18	00011000	CAN	56	38	00111000	8	88	58	01011000	X	120	78	01111000	x
25	19	00011001	EM	57	39	00111001	9	89	59	01011001	Y	121	79	01111001	y
26	1A	00011010	SUB	58	3A	00111010	:	90	5A	01011010	Z	122	7A	01111010	z
27	1B	00011011	ESC	59	3B	00111011	;	91	5B	01011011	[	123	7B	01111011	{
28	1C	00011100	FS	60	3C	00111100	<	92	5C	01011100	\	124	7C	01111100	
29	1D	00011101	GS	61	3D	00111101	=	93	5D	01011101	]	125	7D	01111101	}
30	1E	00011110	RS	62	3E	00111110	>	94	5E	01011110	^	126	7E	01111110	~
31	1F	00011111	US	63	3F	00111111	?	95	5F	01011111	_	127	7F	01111111	DEL